CLOUD

Cloud Architecture Patterns

The building blocks of reliable cloud systems: regions, scaling, and surviving failure.

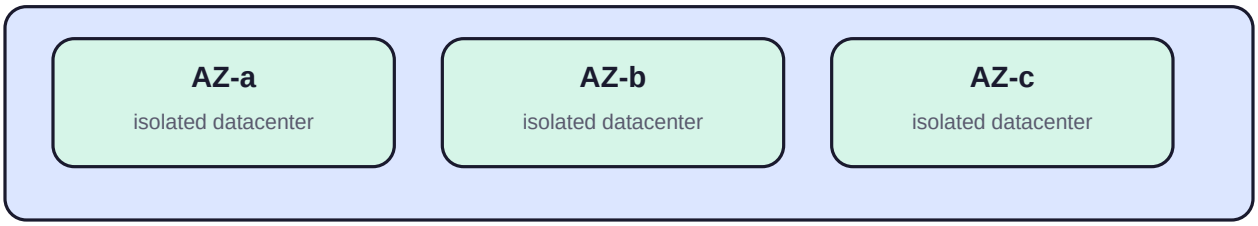
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Regions & Availability Zones

Geography is the first layer of reliability.



AZs: separate power, cooling, networking — a fire in one doesn't touch the others.

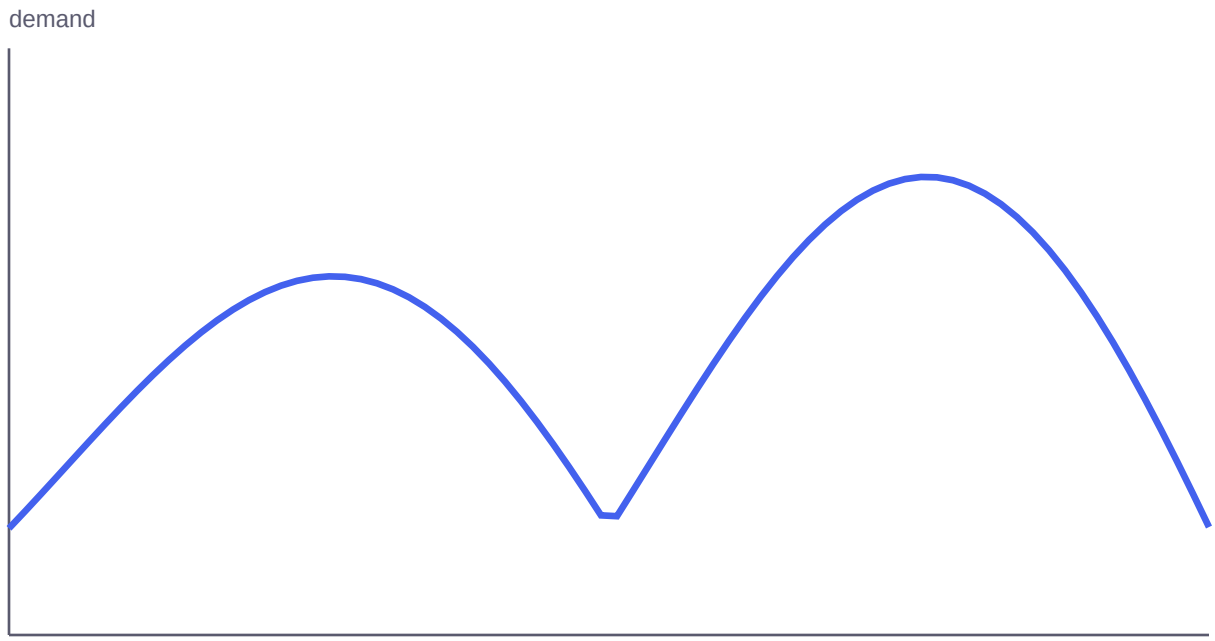
Deploy across ≥2 AZs; replicate data across regions for disaster recovery.

Rule of thumb: survive one AZ failure with zero downtime; survive one region failure with bounded RTO/RPO.

STEP 2/6

Auto Scaling: Match Capacity to Demand

Add machines when busy, remove them when idle — automatically.



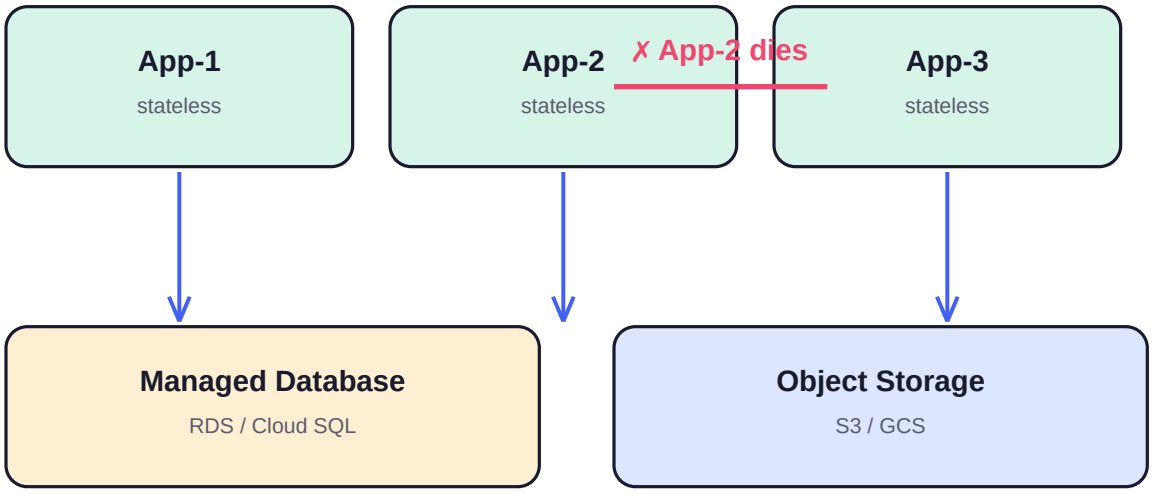
Scaling policies: target-tracking (keep CPU at 60%), step scaling, scheduled (known peaks).

Cooldowns prevent flapping — don't scale in and out on every blip.

STEP 3/6

Stateless Services + External State

Any instance can die; the system doesn't care.

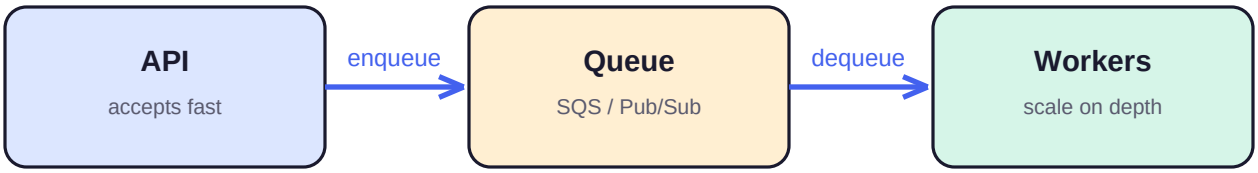


Sessions, uploads, and state live outside the instances — in managed stores.

STEP 4/6

Decouple with Queues

Buffers absorb spikes; workers scale independently.



Traffic spike? The queue grows; workers scale up; nothing is dropped.

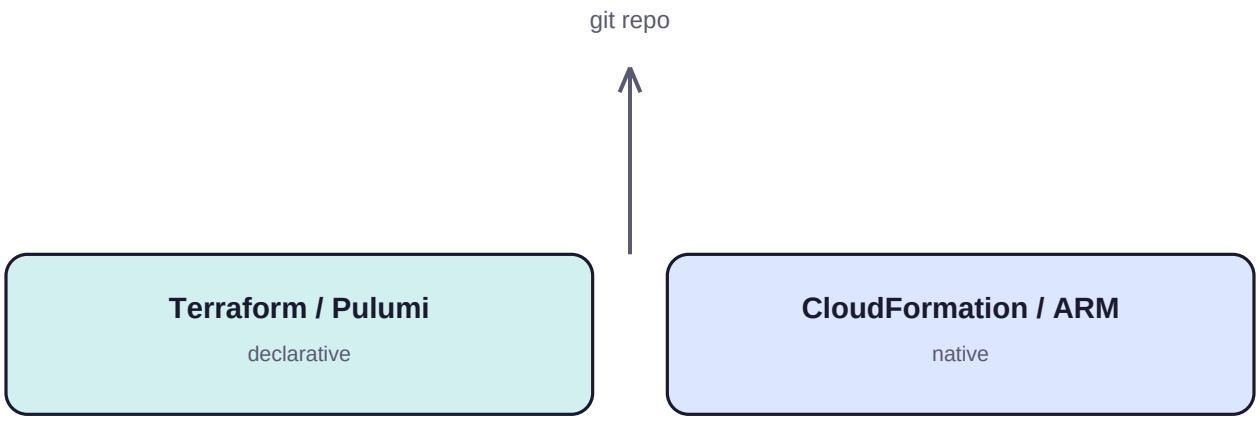
Dead-letter queues capture poison messages for inspection instead of infinite retry.

Queue depth is the autoscaling signal. This is the backbone of event-driven architecture.

STEP 5/6

Infrastructure as Code

Environments defined in files — versioned, reviewed, reproducible.



- plan → review → apply: changes are previewed like code diffs
- State files track reality; drift detection flags manual changes
- Modules = reusable components (a VPC module, a DB module)

STEP 6/6

Well-Architected: The Checklist

Five pillars every design gets judged against.

Reliability

Survive failures; recover fast

Security

Least privilege, encryption everywhere

Cost

Right-size, kill idle, use commitments wisely

Performance

Scale horizontally; cache aggressively

Operations

Automate everything; observe everything

STEP ✓/✓

Key Takeaways

Cloud Patterns — Remember

- Spread across AZs; replicate across regions. Geography is reliability.
- Auto-scale on real signals (CPU, queue depth) with cooldowns against flapping.
- Keep services stateless; push state to managed databases and object storage.
- Queues decouple producers from consumers and absorb spikes gracefully.
- Define infrastructure as code — versioned, reviewed, reproducible.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev